Homework 4: Requirement Elicitation

Group 5: Rui Zhang, Shunkang Jia, Yuning Liu

September 30, 2025

1 Subsystem Decomposition

1.1 Client Management Subsystem (CMS)

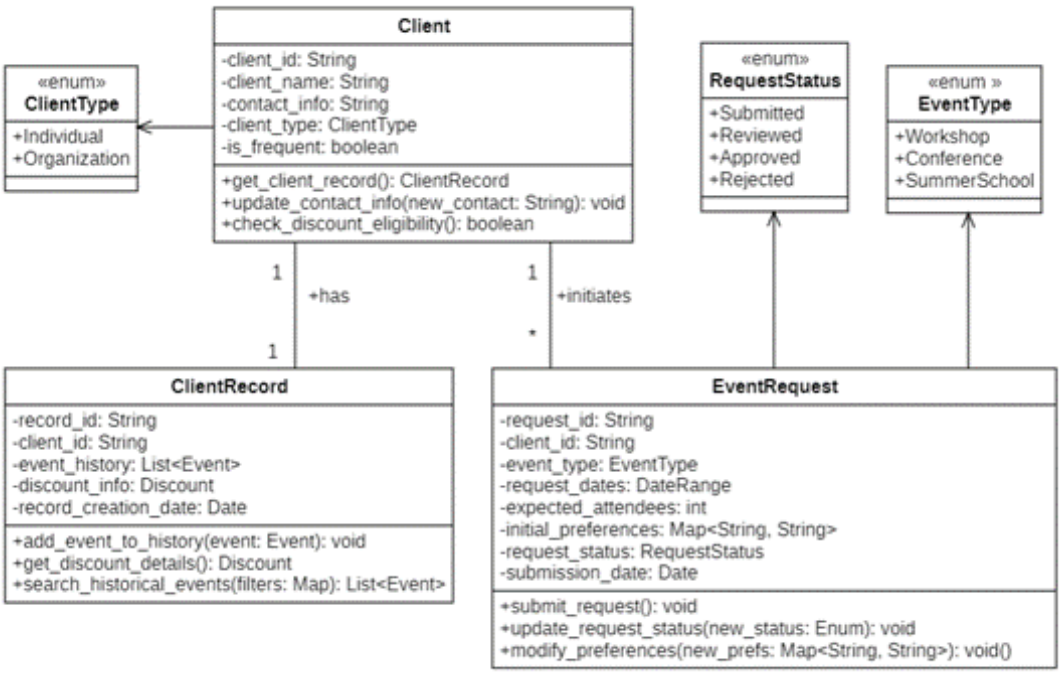

Figure 1: Client Management Subsystem (CMS)

1.2 HR Management Subsystem (HRMS)

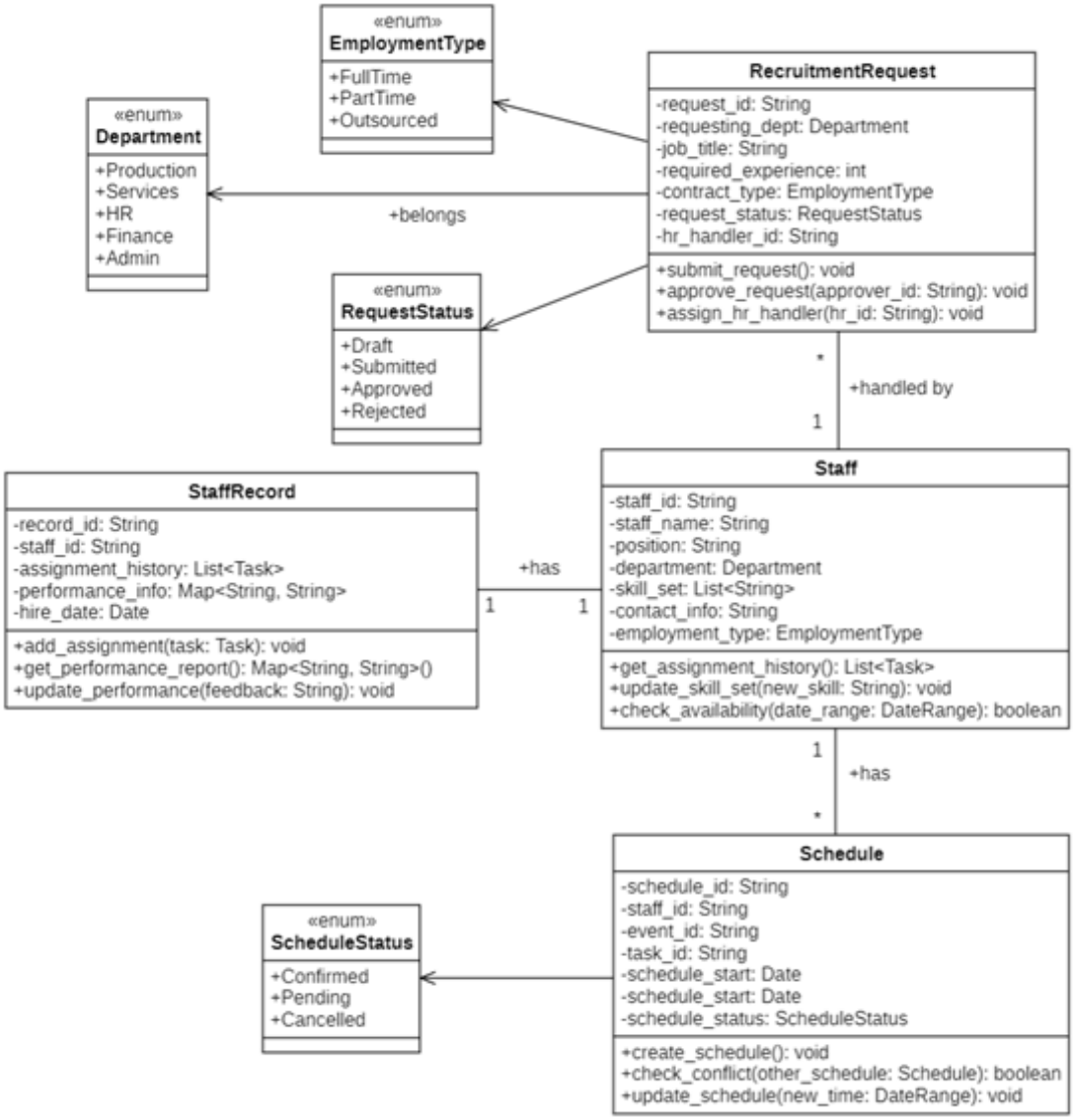

Figure 2: HR Management Subsystem (HRMS)

1.3 Event & Task Management Subsystem (ETMS)

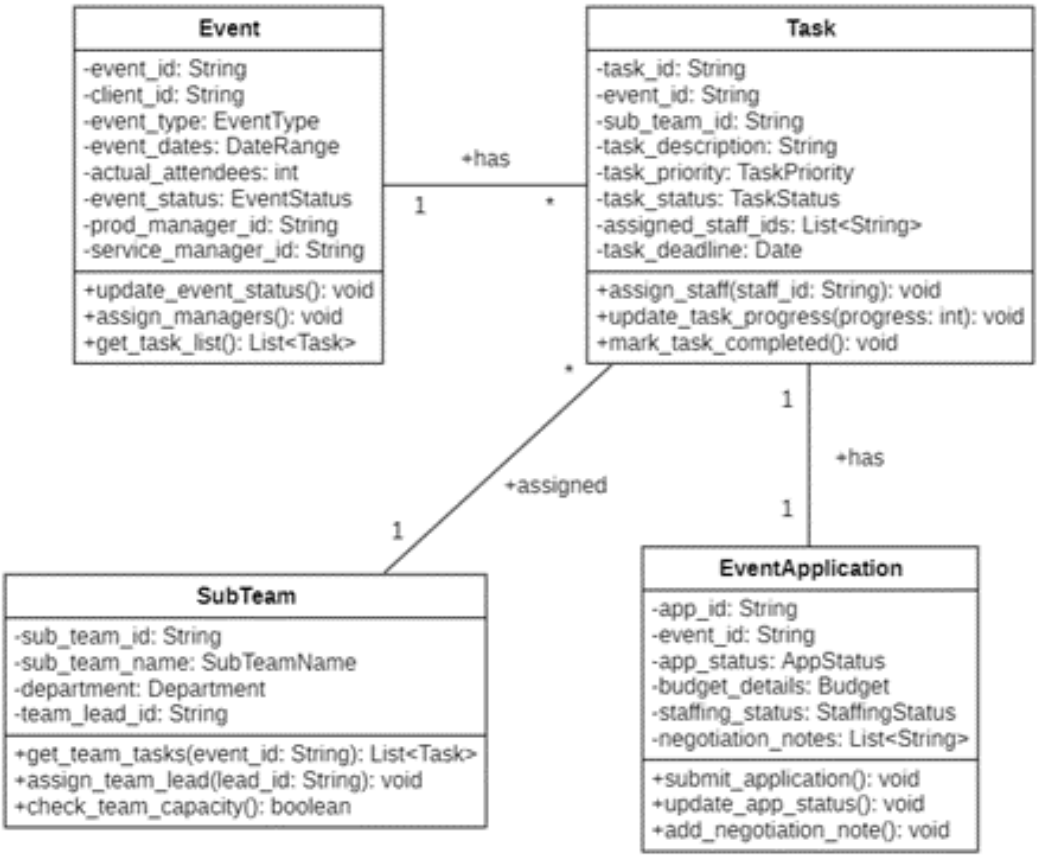

Figure 3: Event & Task Management Subsystem (ETMS)

1.4 Financial Management Subsystem (FMS)

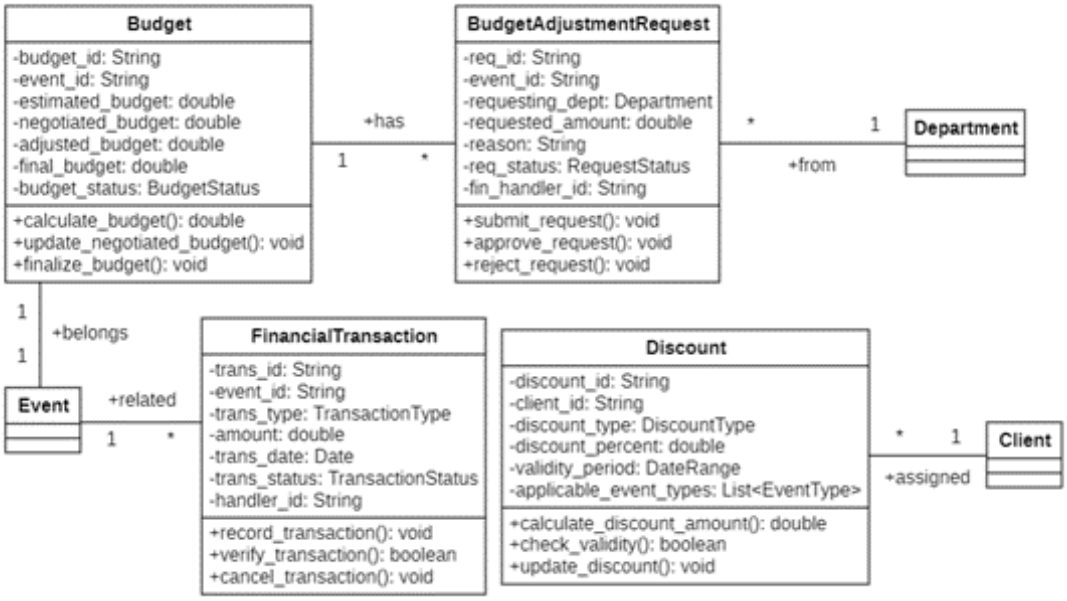

Figure 4: Financial Management Subsystem (FMS)

1.5 Report & Administration Subsystem (RAS)

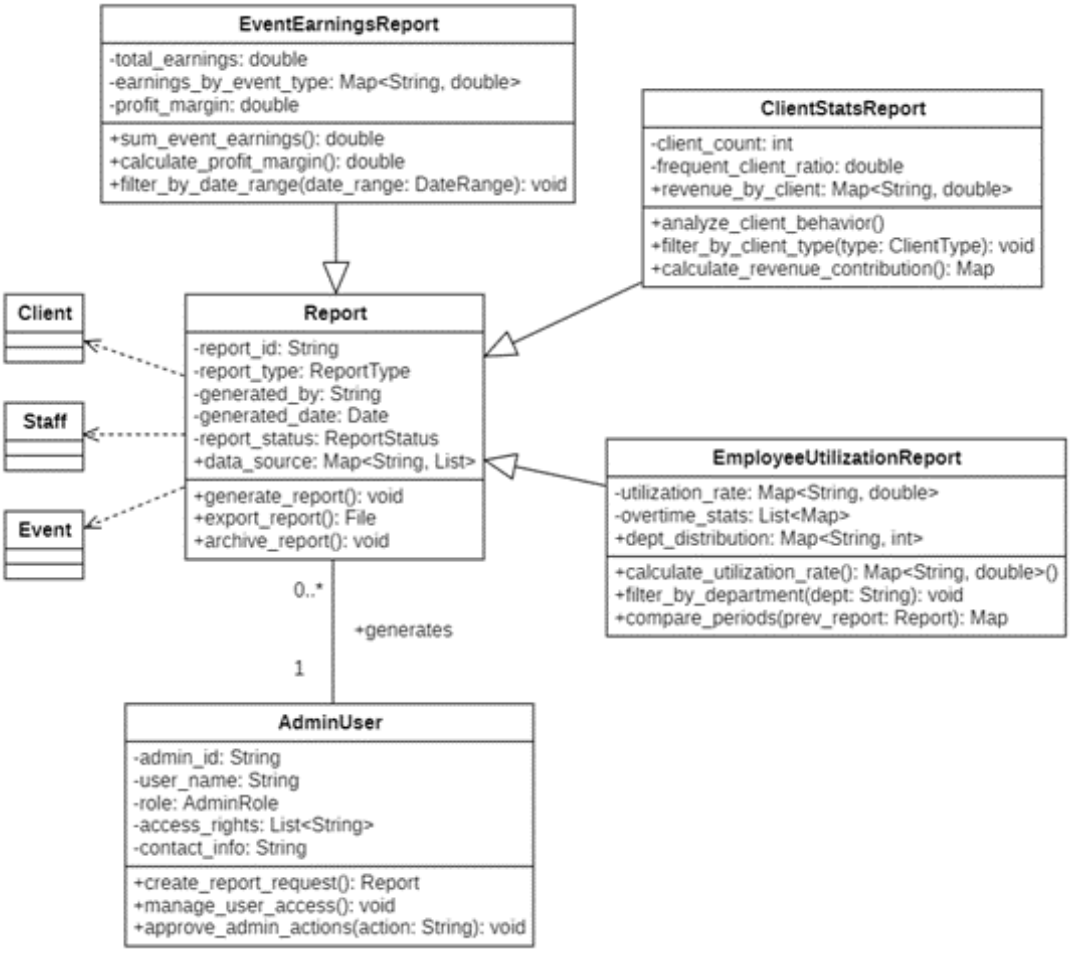

Figure 5: Report & Administration Subsystem (RAS)

2 Deployment

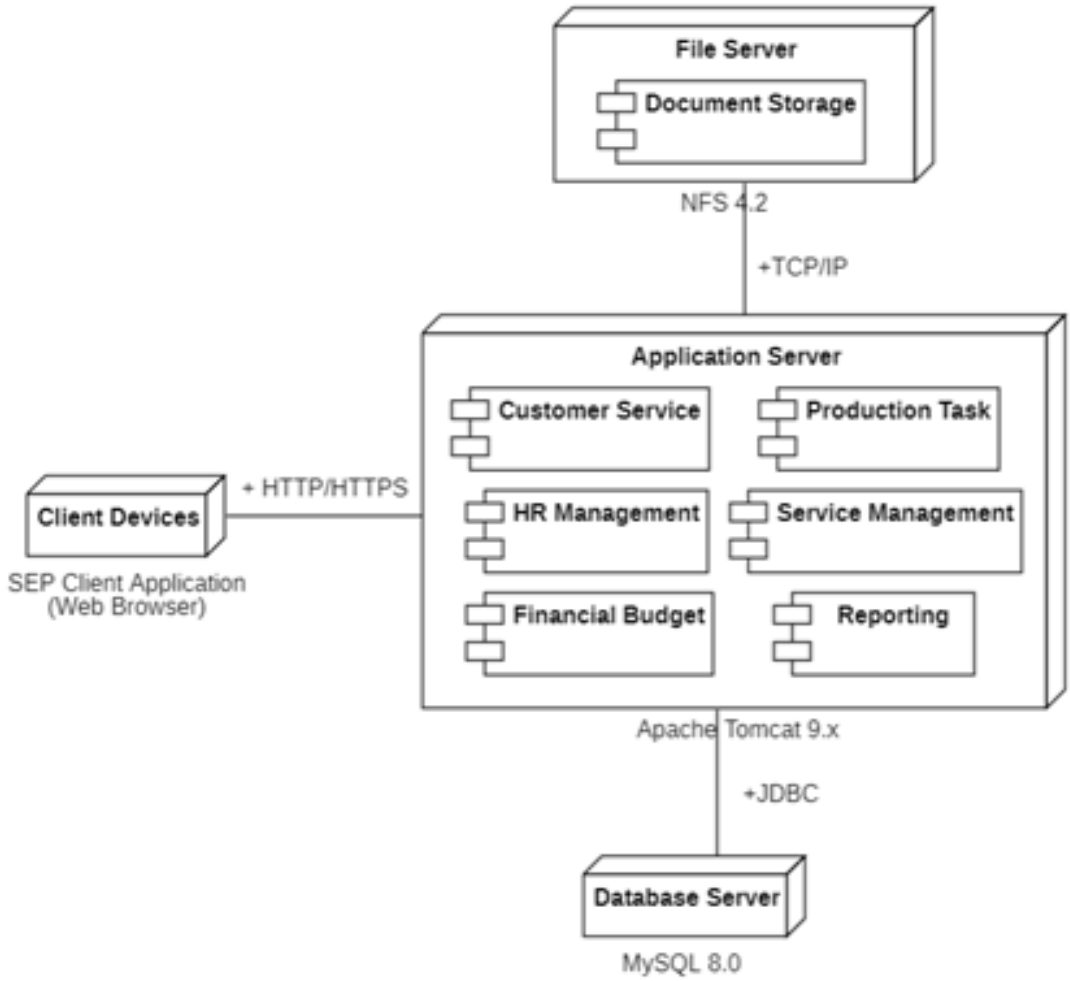

Figure 6: UML Depolyment Graph For SEP Intra System

The system deployment includes four core hardware nodes and corresponding software components:

1. Client devices (employees' PCs/laptops) running web browsers (e.g., Chrome 120+), allowing roles to access the SEP client application;

2. Application server (with Apache Tomcat 9.x), deploying 7 business components—**Customer Service Component** handles event request entry and flow, HR Management Component processes recruitment/outsourcing requests and employee records, **Financial Budget Component** manages budget negotiation and salaries, **Production/Service Task Components** assign sub-team tasks respectively, **Reporting**

Component generates utilization reports for VPs, and **Access Control Component** (Spring Security 6.x) controls role access scope;

3. **Database server** (MySQL 8.0), interacting with the application server via JDBC protocol, storing structured data like clients, employees, and events;

4. **File server** (NFS 4.2), transferring files via TCP/IP, storing unstructured data such as event forms and activity photos. All nodes communicate via HTTP/HTTPS or TCP/IP to ensure secure data transmission and business collaboration.

3 Persistent storage solution

3.1 List of Persistent Objects

The following objects have been identified as requiring persistence:

- **Client**: Each client record is associated with the details of all events for further analyzing, discounting or marketing purpose.

- **Event Planning Request**: It tracks the workflow and encapsulates all information related to a single event. The system is required to keep a complete history of all organized events.

- **Employee**: Including their personal information, department, and a history of their project assignments. This data is used by managers for resource planning and viewing assignment histories.

- **Task**: Task must be persistent as sub-team members need to access, edit, and update them with their plans and comments, such as requests for an extra budget.

- **HR Request**

- **Budget Adjustment Request**

3.2 Description of Storage Management Strategy

We adopted a mixed approach of both file and database.

Considering the complexity and collaborative nature of the SEP workflow which invovles a lot of concurrent queries, we have selected a **Relational Database Management System (RDBMS)** as persistent storage.

To maximize long-term flexibility and reduce development risks, we will adopt an abstraction strategy. Instead of writing code tied to a specific database product, we will use a **database-independent API (e.g., JDBC) or an Object-Relational Mapping (ORM) framework** to interact with the database.

Furthermore, we choose to store all reports (client, staff and event) in a file server, because reports are more flexible.

4 Access Control for SEP System

4.1 Access Control for SEP System

The roles (actors) are identified from the system's user base, and the objects represent the core persistent data entities discussed earlier. This matrix provides a compact and clear overview for reviewing and implementing security policies.

Actors	**Client**	**Event Planning Request**	**STAFF**	**Task**	**HR Request**	**Budget Adjustment Request**
Customer Service Officer		Create/Redirect				
Senior Customer Service Officer	Create/Search/View	Search/View Update/Redirect				
Administration Manager	View	View Approve/Reject	Search/View			
Financial Manager	Search/View Set Discount	View Update Budget`	Search/View		View Approve/Reject	
Operation Manager *(Production/Service)*		View Update Status`	Search/View	create	Create	create
Operation Staff *(Sub-teams)*				View Update Edit Plan ViewSchedule		
HR Team			Search/View		View Update Status	
Marketing Team	Search/View	View (for reports)				
Vice President	View (for reports)	View (for reports)	View (for reports)			

Figure 7: Access Table for SEP System

4.2 Global Control Flow

The global control flow for the SEP system will be a **combination of an event-driven and a threaded control flow paradigm**. This choice is highly suitable for a multi-user, web-based internal management system where multiple actors from different departments need to interact with the system concurrently.

The control flow will operate as follows:

1. **Event-Driven Core**: The system's web server will be event-driven. It will remain in a passive state, waiting for incoming HTTP requests from the web browsers of the users. Each user action that sends data to the server is treated as an **event** that triggers a response.

2. **Threaded Request Handling**: Upon receiving a request, the web server will allocate a new thread to handle that specific request. This allows for the parallel handling of multiple user actions.

This architectural choice is deliberately made to meet key system requirements:

- **Responsiveness and Performance**: This model directly supports the nonfunctional requirement that production and service teams can work in parallel, viewing and editing applications at the same time. By assigning each request its own thread, a long-running operation initiated by one user (e.g., generating a complex report) will not block the entire system, ensuring that other users experience a responsive interface. This improves both system responsiveness and overall throughput.

- **Concurrency Management**: The primary drawback of a threaded model is the complexity of managing concurrent access to shared data. To ensure the system remains robust and data integrity is maintained, a strict synchronization strategy will be implemented, modeled after the ARENA case study:

  - **Stateless Boundary Objects**: Interface-level objects that handle user requests will not hold state in fields, preventing concurrency hazards between different threads.
  - **Synchronized Entity Objects**: All access to the state of persistent entity objects (like `EventPlanningRequest` or `Task`) will be managed through `synchronized` methods. This ensures that only one thread can modify an object's state at any given time, preventing data corruption.
  - **Deadlock Prevention**: Care will be taken to avoid nested calls to synchronized methods to prevent potential system deadlocks.

4.3 Boundary Conditions for the SEP System

We identify the boundary use cases for the SEP system by reviewing its persistent objects, run-time components, and potential failure scenarios. These use cases are primarily intended for an administrative role, such as the **Admin** or a dedicated system operator.

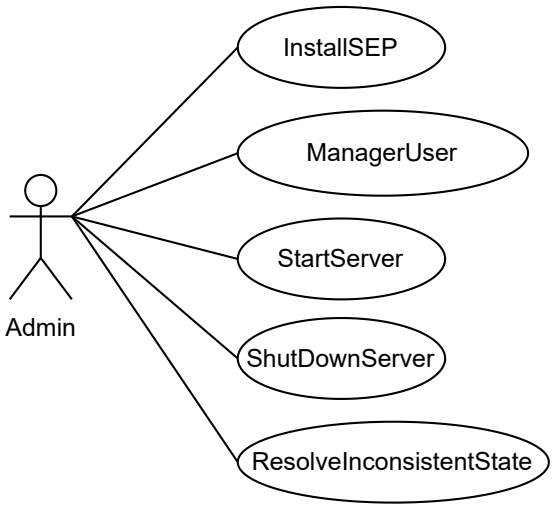

Figure 8: Boundary Use Cases

4.3.1 Configuration Use Cases

These use cases address the setup and long-term management of the SEP system's core components.

Use Case Name	Description
InstallSEP	The Administration Manager performs the initial setup of the SEP system. This includes configuring the persistent storage (e.g., providing database connection details, create tables with admin account)
ManageUser	The Administration Manager or HR Manager creates, modifies, and deactivates user accounts for employees. This use case also involves assigning users to specific roles (e.g., "Financial Manager," "Production Manager") which determines their access rights within the system.

4.3.2 Start-up and Shutdown Use Cases

These use cases describe the controlled initiation and termination of the SEP system's server component.

Use Case Name	Description
StartServer	The Administration Manager starts the main SEP application server. If the server detects that it was not shut down cleanly during its previous run. Once initialization is complete, employees can log in and perform their tasks.
ShutDownServer	The Administration Manager gracefully stops the SEP server. The system prevents new user logins, waits for any active operations to complete, writes all cached data to the persistent store, and closes database connections. This ensures no data is lost during planned maintenance or system updates.

4.3.3 Exception Use Cases

These use cases define how the system handles major failures to ensure data integrity and system reliability.

Use Case Name	Description
ResolveInconsistentState	Following a crash, some business objects might be left in an inconsistent state (e.g., a `Task` is assigned but the parent `EventPlanningRequest` is not set to "in progress"). This use case allows the Administration Manager to manually review flagged inconsistencies and correct the state of objects in the database to restore normal business flow.

5 Design Patterns

5.1 Proxy Pattern

The access control is an essential part of the SEP system, and we decide to adopt proxy pattern to implement the access control.

For example, the staff record is only available to HR, Financial Manager and Vice President, we can implement `StaffProxyService` class and examine if the user has the access to the record then decide whether to call the real `StaffService`.

5.2 Observer Pattern

The status of an `EventPlanningRequest` is a central piece of information that drives the entire workflow, changing from "open" to "in progress" and finally to "closed and archived". Multiple actors and parts of the system need to react to these state changes. Instead of checking status change in each of these listeners, we decide to apply observer pattern.

The `EventPlanningRequest` object will act as the "Subject." Other objects, such as a `ProductionManager` or a `NotificationService`, will act as "Observers." These observers will register themselves with the `EventPlanningRequest` object they are interested in. When the `setStatus()` method is called on the `EventPlanningRequest`, it will not only change its own state but also iterate through its list of registered observers and call a `notify()` or `update()` method on each one, passing along information about the state change.

6 Contracts in OCL

6.1 Client Management

A discount can only be applied to an event request if the associated client is a frequent client.
context FinancialManager::makeDiscount(request : EventPlanningRequest)
pre: request.client.isFrequentClient = true

An event planning request can only be redirected to the Administration Manager for a final decision after the Financial Manager has provided feedback.
context SeniorCustomerServiceOfficer::redirectForAdminApproval(request : EventPlanningRequest)
pre: request.financialFeedback→notEmpty()

A client's contact information (e.g., email) must be unique across the entire system to prevent duplicate records. This is an inferred data integrity constraint for any client management system.
context Client
inv: Client.allInstances()→forAll(c1, c2 | c1 <> c2 implies c1.email <> c2.email)

6.2 HR Management

An HR request for a specific event can only be marked as 'Fulfilled' after the newly acquired staff member is formally assigned to that event's team.
context HRTeam::fulfillRequest(hrRequest : HRRequest, newEmployee : Employee)
post: hrRequest.status = 'Fulfilled' and hrRequest.event.assignedStaff.includes(newEmployee)

An employee's recorded role must match one of the official job titles defined within their assigned department.
context Employee
inv: self.department.validJobTitles→ includes(self.role)

6.3 Event & Task Management

An employee cannot be assigned to two tasks with overlapping time schedules. This enforces logical resource scheduling, building upon the rule that staff can work on multiple projects.

context Employee
inv: self.assignedTasks→forAll(t1, t2 | t1 <> t2 implies t1.endDate.isBefore(t2.startDate) or t1.startDate.isAfter(t2.endDate))

An EventPlanningRequest can only be set to 'closed' status if all of its associated tasks have been marked as 'completed'. This enforces the rule that an event is finalized only after all its work is done.

context OperationManager::archiveEvent(request : EventPlanningRequest)
pre: request.tasks→forAll(t | t.status = 'completed')

6.4 Financial Management

The total budget approved for an event must be greater than or equal to the sum of the budgets of all its individual tasks.

context EventPlanningRequest
inv: self.budget >= self.tasks→collect(budget)→sum()

The system must halt the processing of an event request until all budget issues are resolved with the client.

context OperationManager::beginEventExecution(request : EventPlanningRequest)
pre: request.budgetIssuesSolved = true

6.5 Report & Administration

The Administration Manager can only make a final decision (approve/reject) on a request after the Financial Manager has provided feedback. This precondition enforces the established decision-making hierarchy.

context AdministrationManager::makeFinalDecision(request : EventPlanningRequest)
pre: request.financialFeedback→notEmpty()